

ankicpp

Generated by Doxygen 1.9.8

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 ankippp::Card Class Reference	7
4.2 ankippp::CardDTO Class Reference	8
4.3 ankippp::CardRepository Class Reference	9
4.4 ankippp::ColDTO Class Reference	10
4.5 ankippp::ColRepository Class Reference	10
4.6 ankippp::ConfigDTO Class Reference	11
4.7 ankippp::ConfigRepository Class Reference	12
4.8 ankippp::Deck Class Reference	13
4.9 ankippp::DeckConfigDTO Class Reference	14
4.10 ankippp::DeckConfigRepository Class Reference	15
4.11 ankippp::DeckDTO Class Reference	16
4.12 ankippp::DeckRepository Class Reference	16
4.13 ankippp::Field Class Reference	17
4.14 ankippp::FieldDTO Class Reference	19
4.15 ankippp::FieldRepository Class Reference	19
4.16 ankippp::GraveDTO Class Reference	20
4.17 ankippp::Identifiable Class Reference	20
4.18 ankippp::Nameable Class Reference	21
4.19 ankippp::Note Class Reference	22
4.20 ankippp::NoteDTO Class Reference	23
4.21 ankippp::NoteRepository Class Reference	24
4.22 ankippp::NoteType Class Reference	25
4.23 ankippp::NoteTypeDTO Class Reference	26
4.24 ankippp::NoteTypeRepository Class Reference	27
4.25 ankippp::Repository< D, K > Class Template Reference	28
4.26 ankippp::Repository< CardDTO, std::int64_t > Class Reference	28
4.27 ankippp::Repository< ColDTO, std::int64_t > Class Reference	29
4.28 ankippp::Repository< ConfigDTO, std::int64_t > Class Reference	29
4.29 ankippp::Repository< DeckConfigDTO, std::int64_t > Class Reference	30
4.30 ankippp::Repository< DeckDTO, std::int64_t > Class Reference	31
4.31 ankippp::Repository< FieldDTO, std::int64_t > Class Reference	31
4.32 ankippp::Repository< NoteDTO, std::int64_t > Class Reference	32
4.33 ankippp::Repository< NoteTypeDTO, std::int64_t > Class Reference	33

4.34 ankipcpp::Repository< TemplateDTO, std::int64_t > Class Reference	33
4.35 ankipcpp::TagDTO Class Reference	34
4.36 ankipcpp::Template Class Reference	34
4.37 ankipcpp::TemplateDTO Class Reference	36
4.38 ankipcpp::TemplateRepository Class Reference	36
4.39 soci::type_conversion< ankipcpp::CardDTO > Struct Reference	37
4.40 soci::type_conversion< ankipcpp::ColDTO > Struct Reference	38
4.41 soci::type_conversion< ankipcpp::ConfigDTO > Struct Reference	38
4.42 soci::type_conversion< ankipcpp::DeckConfigDTO > Struct Reference	38
4.43 soci::type_conversion< ankipcpp::DeckDTO > Struct Reference	39
4.44 soci::type_conversion< ankipcpp::FieldDTO > Struct Reference	39
4.45 soci::type_conversion< ankipcpp::NoteDTO > Struct Reference	39
4.46 soci::type_conversion< ankipcpp::NoteTypeDTO > Struct Reference	40
4.47 soci::type_conversion< ankipcpp::TemplateDTO > Struct Reference	40
5 File Documentation	41
5.1 ankipcpp.h	41
5.2 export.h	41
5.3 card-dto.h	42
5.4 card-mapper.h	42
5.5 card-repository.h	42
5.6 card.h	42
5.7 col-dto.h	43
5.8 col-repository.h	43
5.9 col.h	44
5.10 compression.h	44
5.11 zip-utils.h	44
5.12 zstd-utils.h	44
5.13 config-dto.h	44
5.14 config-repository.h	46
5.15 config.h	46
5.16 database.h	46
5.17 repository.h	47
5.18 deck-config-dto.h	47
5.19 deck-config-repository.h	47
5.20 deck-dto.h	48
5.21 deck-mapper.h	48
5.22 deck-repository.h	48
5.23 deck.h	49
5.24 field-dto.h	49
5.25 field-mapper.h	49
5.26 field-repository.h	50

5.27 field.h	50
5.28 grave-dto.h	50
5.29 grave.h	50
5.30 note-dto.h	51
5.31 note-mapper.h	51
5.32 note-repository.h	51
5.33 note-type-dto.h	51
5.34 note-type-mapper.h	52
5.35 note-type-repository.h	52
5.36 note-type.h	52
5.37 note.h	53
5.38 reflog-dto.h	53
5.39 reflog.h	54
5.40 tag-dto.h	54
5.41 tag.h	54
5.42 template-dto.h	54
5.43 template-mapper.h	54
5.44 template-repository.h	55
5.45 template.h	55
5.46 identifiable.h	55
5.47 nameable.h	56
5.48 time.h	56
5.49 util.h	56
Index	57

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

ankicpp::CardDTO	8
ankicpp::ColDTO	10
ankicpp::ConfigDTO	11
ankicpp::DeckConfigDTO	14
ankicpp::DeckDTO	16
std::enable_shared_from_this	
ankicpp::NoteType	25
ankicpp::FieldDTO	19
ankicpp::GraveDTO	20
ankicpp::Identifiable	20
ankicpp::Card	7
ankicpp::Deck	13
ankicpp::Field	17
ankicpp::Note	22
ankicpp::NoteType	25
ankicpp::Template	34
ankicpp::Nameable	21
ankicpp::Deck	13
ankicpp::Field	17
ankicpp::NoteType	25
ankicpp::Template	34
ankicpp::NoteDTO	23
ankicpp::NoteTypeDTO	26
ankicpp::Repository< D, K >	28
ankicpp::Repository< CardDTO, std::int64_t >	28
ankicpp::CardRepository	9
ankicpp::Repository< ColDTO, std::int64_t >	29
ankicpp::ColRepository	10
ankicpp::Repository< ConfigDTO, std::int64_t >	29
ankicpp::ConfigRepository	12
ankicpp::Repository< DeckConfigDTO, std::int64_t >	30
ankicpp::DeckConfigRepository	15
ankicpp::Repository< DeckDTO, std::int64_t >	31

ankicpp::DeckRepository	16
ankicpp::Repository< FieldDTO, std::int64_t >	31
ankicpp::FieldRepository	19
ankicpp::Repository< NoteDTO, std::int64_t >	32
ankicpp::NoteRepository	24
ankicpp::Repository< NoteTypeDTO, std::int64_t >	33
ankicpp::NoteTypeRepository	27
ankicpp::Repository< TemplateDTO, std::int64_t >	33
ankicpp::TemplateRepository	36
ankicpp::TagDTO	34
ankicpp::TemplateDTO	36
soci::type_conversion< ankicpp::CardDTO >	37
soci::type_conversion< ankicpp::ColDTO >	38
soci::type_conversion< ankicpp::ConfigDTO >	38
soci::type_conversion< ankicpp::DeckConfigDTO >	38
soci::type_conversion< ankicpp::DeckDTO >	39
soci::type_conversion< ankicpp::FieldDTO >	39
soci::type_conversion< ankicpp::NoteDTO >	39
soci::type_conversion< ankicpp::NoteTypeDTO >	40
soci::type_conversion< ankicpp::TemplateDTO >	40

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ankicpp::Card	7
ankicpp::CardDTO	8
ankicpp::CardRepository	9
ankicpp::ColDTO	10
ankicpp::ColRepository	10
ankicpp::ConfigDTO	11
ankicpp::ConfigRepository	12
ankicpp::Deck	13
ankicpp::DeckConfigDTO	14
ankicpp::DeckConfigRepository	15
ankicpp::DeckDTO	16
ankicpp::DeckRepository	16
ankicpp::Field	17
ankicpp::FieldDTO	19
ankicpp::FieldRepository	19
ankicpp::GraveDTO	20
ankicpp::Identifiable	20
ankicpp::Nameable	21
ankicpp::Note	22
ankicpp::NoteDTO	23
ankicpp::NoteRepository	24
ankicpp::NoteType	25
ankicpp::NoteTypeDTO	26
ankicpp::NoteTypeRepository	27
ankicpp::Repository< D, K >	28
ankicpp::Repository< CardDTO, std::int64_t >	28
ankicpp::Repository< ColDTO, std::int64_t >	29
ankicpp::Repository< ConfigDTO, std::int64_t >	29
ankicpp::Repository< DeckConfigDTO, std::int64_t >	30
ankicpp::Repository< DeckDTO, std::int64_t >	31
ankicpp::Repository< FieldDTO, std::int64_t >	31
ankicpp::Repository< NoteDTO, std::int64_t >	32
ankicpp::Repository< NoteTypeDTO, std::int64_t >	33
ankicpp::Repository< TemplateDTO, std::int64_t >	33
ankicpp::TagDTO	34

ankicpp::Template	34
ankicpp::TemplateDTO	36
ankicpp::TemplateRepository	36
soci::type_conversion< ankicpp::CardDTO >	37
soci::type_conversion< ankicpp::CoIDTO >	38
soci::type_conversion< ankicpp::ConfigDTO >	38
soci::type_conversion< ankicpp::DeckConfigDTO >	38
soci::type_conversion< ankicpp::DeckDTO >	39
soci::type_conversion< ankicpp::FieldDTO >	39
soci::type_conversion< ankicpp::NoteDTO >	39
soci::type_conversion< ankicpp::NoteTypeDTO >	40
soci::type_conversion< ankicpp::TemplateDTO >	40

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

src/ankicpp/ankicpp.h	41
src/ankicpp/export.h	41
src/card/card-dto.h	42
src/card/card-mapper.h	42
src/card/card-repository.h	42
src/card/card.h	42
src/col/col-dto.h	43
src/col/col-repository.h	43
src/col/col.h	44
src/compression/compression.h	44
src/compression/zip-utils.h	44
src/compression/zstd-utils.h	44
src/config/config-dto.h	44
src/config/config-repository.h	46
src/config/config.h	46
src/database/database.h	46
src/database/repository.h	47
src/deck/deck-config-dto.h	47
src/deck/deck-config-repository.h	47
src/deck/deck-dto.h	48
src/deck/deck-mapper.h	48
src/deck/deck-repository.h	48
src/deck/deck.h	49
src/field/field-dto.h	49
src/field/field-mapper.h	49
src/field/field-repository.h	50
src/field/field.h	50
src/grave/grave-dto.h	50
src/grave/grave.h	50
src/note/note-dto.h	51
src/note/note-mapper.h	51
src/note/note-repository.h	51
src/note/note-type-dto.h	51
src/note/note-type-mapper.h	52
src/note/note-type-repository.h	52

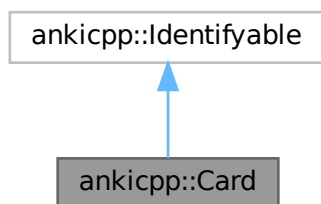
src/note/ note-type.h	52
src/note/ note.h	53
src/reflog/ reflog-dto.h	53
src/reflog/ reflog.h	54
src/tag/ tag-dto.h	54
src/tag/ tag.h	54
src/template/ template-dto.h	54
src/template/ template-mapper.h	54
src/template/ template-repository.h	55
src/template/ template.h	55
src/util/ identifiable.h	55
src/util/ nameable.h	56
src/util/ time.h	56
src/util/ util.h	56

Chapter 4

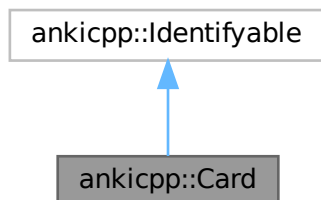
Class Documentation

4.1 ankippp::Card Class Reference

Inheritance diagram for ankippp::Card:



Collaboration diagram for ankippp::Card:



Public Member Functions

- `std::shared_ptr< Deck > getDeck ()` const
- `void setDeck (std::shared_ptr< Deck > deck)`
- `std::shared_ptr< Template > getTemplate ()` const
- `void setTemplate (std::shared_ptr< Template > cardType)`
- `std::shared_ptr< Note > getNote ()` const
- `void setNote (std::shared_ptr< Note > note)`

Public Member Functions inherited from [ankicpp::Identifiable](#)

- `std::int64_t getId ()`
- `void setId (std::int64_t id)`
- `bool operator== (Identifiable &rhs)`
- `bool operator!= (Identifiable &rhs)`
- `bool operator< (Identifiable &rhs)`

Additional Inherited Members

Static Public Member Functions inherited from [ankicpp::Identifiable](#)

- `static std::int64_t generateRandomId ()`

Protected Attributes inherited from [ankicpp::Identifiable](#)

- `std::int64_t _id`

The documentation for this class was generated from the following files:

- `src/card/card.h`
- `src/card/card.cc`

4.2 [ankicpp::CardDTO](#) Class Reference

Public Attributes

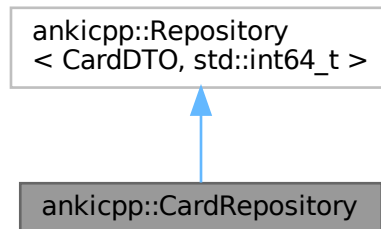
- `std::int64_t id`
- `std::int64_t nid`
- `std::int64_t did`
- `std::int64_t ord`
- `std::int64_t mod`
- `std::int64_t usn`
- `std::int64_t type`
- `std::int64_t queue`
- `std::int64_t due`
- `std::int64_t ivl`
- `std::int64_t factor`
- `std::int64_t reps`
- `std::int64_t lapses`
- `std::int64_t left`
- `std::int64_t odue`
- `std::int64_t odid`
- `std::int64_t flags`
- `std::string data`

The documentation for this class was generated from the following file:

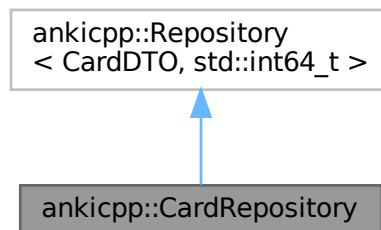
- `src/card/card-dto.h`

4.3 ankippp::CardRepository Class Reference

Inheritance diagram for ankippp::CardRepository:



Collaboration diagram for ankippp::CardRepository:



Additional Inherited Members

Public Member Functions inherited from `ankipp::Repository< CardDTO, std::int64_t >`

- `std::tuple< std::vector< CardDTO >, bool > read () const`
- `std::tuple< CardDTO, bool > readById (std::int64_t key) const`
- `bool create (CardDTO cardDTO) const`
- `bool update (CardDTO cardDTO) const`
- `bool del (CardDTO cardDTO) const`

The documentation for this class was generated from the following file:

- `src/card/card-repository.h`

4.4 ankicpp::ColDTO Class Reference

Static Public Member Functions

- static [ColDTO](#) createDTO ()

Public Attributes

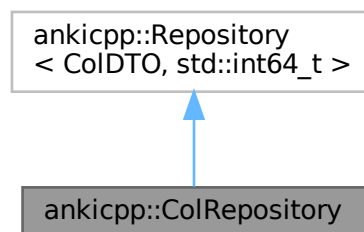
- std::int64_t **id**
- std::int64_t **crt**
- std::int64_t **mod**
- std::int64_t **scm**
- std::int64_t **ver**
- std::int64_t **dtv**
- std::int64_t **usn**
- std::int64_t **ls**
- std::string **conf**
- std::string **models**
- std::string **decks**
- std::string **dconf**
- std::string **tags**

The documentation for this class was generated from the following file:

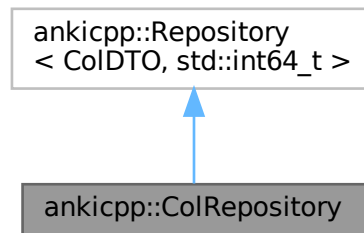
- src/col/col-dto.h

4.5 ankicpp::ColRepository Class Reference

Inheritance diagram for ankicpp::ColRepository:



Collaboration diagram for ankippp::ColRepository:



Additional Inherited Members

Public Member Functions inherited from `ankipp::Repository< ColDTO, std::int64_t >`

- `std::tuple< std::vector< ColDTO >, bool > read () const`
- `std::tuple< ColDTO, bool > readById (std::int64_t key) const`
- `bool create (ColDTO colDTO) const`
- `bool update (ColDTO colDTO) const`
- `bool del (ColDTO colDTO) const`

The documentation for this class was generated from the following file:

- `src/col/col-repository.h`

4.6 ankippp::ConfigDTO Class Reference

Static Public Member Functions

- `static std::vector< ConfigDTO > createDTOs ()`

Public Attributes

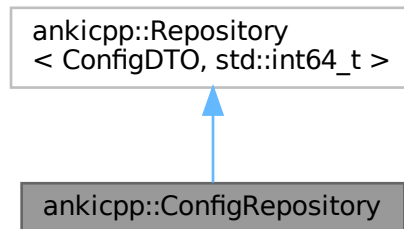
- `std::string KEY`
- `std::int64_t usn`
- `std::int64_t mtime_secs`
- `std::vector< unsigned char > val`

The documentation for this class was generated from the following file:

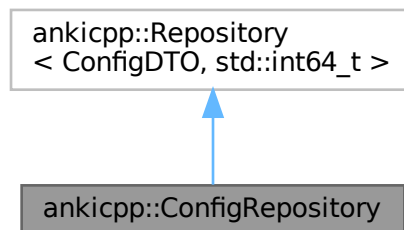
- `src/config/config-dto.h`

4.7 ankippp::ConfigRepository Class Reference

Inheritance diagram for ankippp::ConfigRepository:



Collaboration diagram for ankippp::ConfigRepository:



Additional Inherited Members

Public Member Functions inherited from [ankipp::Repository< ConfigDTO, std::int64_t >](#)

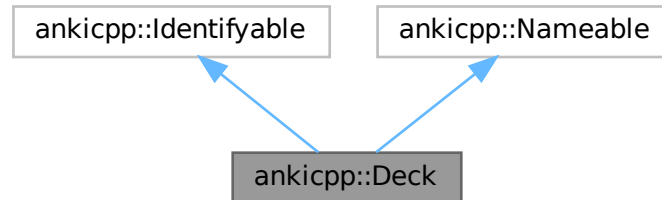
- `std::tuple< std::vector< ConfigDTO >, bool > read () const`
- `std::tuple< ConfigDTO, bool > readById (std::int64_t key) const`
- `bool create (ConfigDTO configDTO) const`
- `bool update (ConfigDTO configDTO) const`
- `bool del (ConfigDTO configDTO) const`

The documentation for this class was generated from the following file:

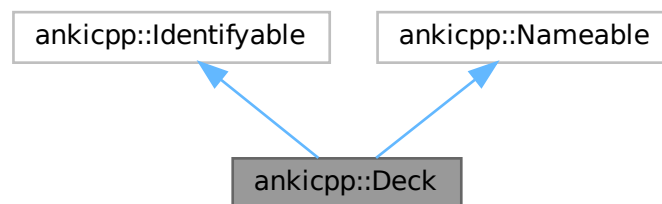
- `src/config/config-repository.h`

4.8 ankippp::Deck Class Reference

Inheritance diagram for ankippp::Deck:



Collaboration diagram for ankippp::Deck:



Public Member Functions

- **Deck** (`std::string` name)
- `std::set< std::shared_ptr< Note > > &getNotes ()`
- `void addNote (std::shared_ptr< Note > note)`
- `void removeNote (std::shared_ptr< Note > note)`
- `std::set< std::shared_ptr< Card > > &getCards ()`
- `void addCard (std::shared_ptr< Card > card)`
- `void removeCard (std::shared_ptr< Card > card)`
- `void clearCards ()`
- `bool generateCards ()`

Public Member Functions inherited from [ankipp::Identifyable](#)

- `std::int64_t getId ()`
- `void setId (std::int64_t id)`
- `bool operator== (Identifyable &rhs)`
- `bool operator!= (Identifyable &rhs)`
- `bool operator< (Identifyable &rhs)`

Public Member Functions inherited from [ankicpp::Nameable](#)

- **Nameable** (std::string name)
- std::string **getName** ()
- void **setName** (std::string name)

Additional Inherited Members

Static Public Member Functions inherited from [ankicpp::Identifiable](#)

- static std::int64_t **generateRandomId** ()

Protected Attributes inherited from [ankicpp::Identifiable](#)

- std::int64_t **_id**

Protected Attributes inherited from [ankicpp::Nameable](#)

- std::string **_name**

The documentation for this class was generated from the following files:

- src/deck/deck.h
- src/deck/deck.cc

4.9 [ankicpp::DeckConfigDTO](#) Class Reference

Static Public Member Functions

- static [DeckConfigDTO](#) **createDefaultDTO** ()

Public Attributes

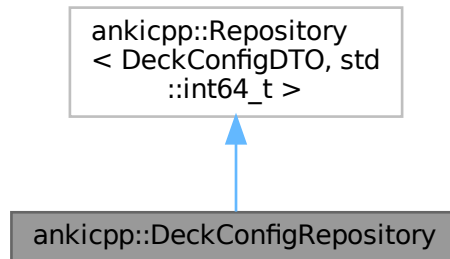
- std::int64_t **id**
- std::string **name**
- std::int64_t **mtime_secs**
- std::int64_t **usn**
- std::vector< unsigned char > **config**

The documentation for this class was generated from the following file:

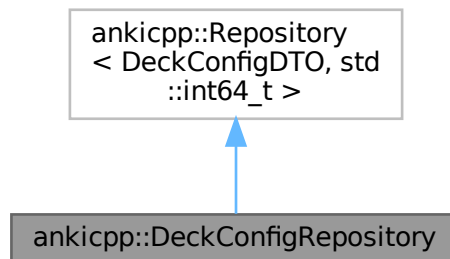
- src/deck/deck-config-dto.h

4.10 ankippp::DeckConfigRepository Class Reference

Inheritance diagram for ankippp::DeckConfigRepository:



Collaboration diagram for ankippp::DeckConfigRepository:



Additional Inherited Members

Public Member Functions inherited from

[`ankipp::Repository< DeckConfigDTO, std::int64\_t >`](#)

- `std::tuple< std::vector< DeckConfigDTO >, bool > read ()` const
- `std::tuple< DeckConfigDTO, bool > readById (std::int64_t key)` const
- `bool create (DeckConfigDTO deckConfigDTO)` const
- `bool update (DeckConfigDTO deckConfigDTO)` const
- `bool del (DeckConfigDTO deckConfigDTO)` const

The documentation for this class was generated from the following file:

- `src/deck/deck-config-repository.h`

4.11 ankicpp::DeckDTO Class Reference

Static Public Member Functions

- static [DeckDTO](#) `createDefaultDTO` ()

Public Attributes

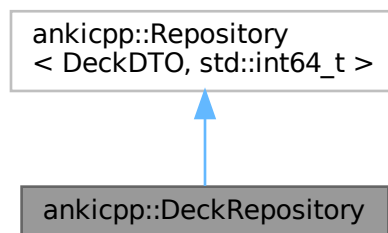
- `std::int64_t` **id**
- `std::string` **name**
- `std::int64_t` **mtime_secs**
- `std::int64_t` **usn**
- `std::vector< unsigned char >` **common**
- `std::vector< unsigned char >` **kind**

The documentation for this class was generated from the following file:

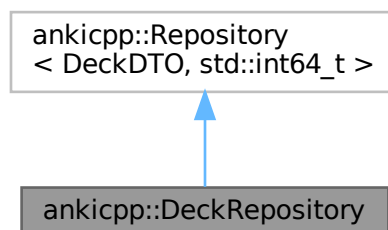
- `src/deck/deck-dto.h`

4.12 ankicpp::DeckRepository Class Reference

Inheritance diagram for `ankicpp::DeckRepository`:



Collaboration diagram for `ankicpp::DeckRepository`:



Additional Inherited Members

Public Member Functions inherited from [ankipp::Repository< DeckDTO, std::int64_t >](#)

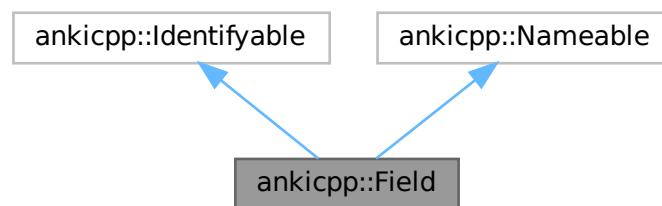
- `std::tuple< std::vector< DeckDTO >, bool > read () const`
- `std::tuple< DeckDTO, bool > readById (std::int64_t key) const`
- `bool create (DeckDTO deckDTO) const`
- `bool update (DeckDTO deckDTO) const`
- `bool del (DeckDTO deckDTO) const`

The documentation for this class was generated from the following file:

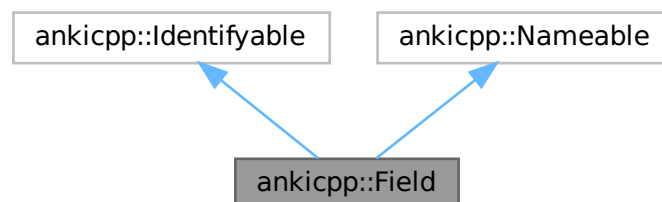
- `src/deck/deck-repository.h`

4.13 ankippp::Field Class Reference

Inheritance diagram for ankippp::Field:



Collaboration diagram for ankippp::Field:



Public Member Functions

- **Field** (std::string name)
- std::weak_ptr< [NoteType](#) > **getNoteType** () const
- void **setNoteType** (std::weak_ptr< [NoteType](#) > noteType)
- std::string **getDescription** () const
- void **setDescription** (std::string description)
- std::string **getFont** () const
- void **setFont** (std::string font)
- uint **getFontSize** () const
- void **setFontSize** (uint fontSize)

Public Member Functions inherited from [ankicpp::Identifiable](#)

- std::int64_t **getId** ()
- void **setId** (std::int64_t id)
- bool **operator==** ([Identifiable](#) &rhs)
- bool **operator!=** ([Identifiable](#) &rhs)
- bool **operator<** ([Identifiable](#) &rhs)

Public Member Functions inherited from [ankicpp::Nameable](#)

- **Nameable** (std::string name)
- std::string **getName** ()
- void **setName** (std::string name)

Additional Inherited Members

Static Public Member Functions inherited from [ankicpp::Identifiable](#)

- static std::int64_t **generateRandomId** ()

Protected Attributes inherited from [ankicpp::Identifiable](#)

- std::int64_t **_id**

Protected Attributes inherited from [ankicpp::Nameable](#)

- std::string **_name**

The documentation for this class was generated from the following files:

- src/field/field.h
- src/field/field.cc

4.14 ankipcpp::FieldDTO Class Reference

Public Attributes

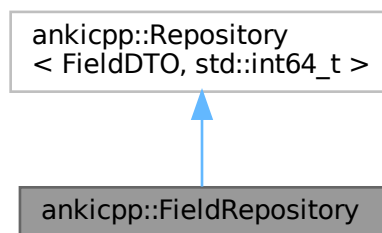
- `std::int64_t` **ntid**
- `std::int64_t` **ord**
- `std::string` **name**
- `std::vector< unsigned char >` **config**

The documentation for this class was generated from the following file:

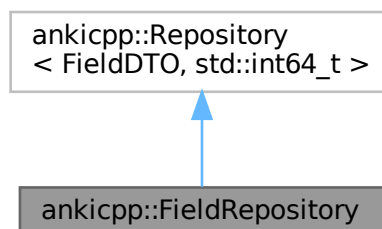
- `src/field/field-dto.h`

4.15 ankipcpp::FieldRepository Class Reference

Inheritance diagram for ankipcpp::FieldRepository:



Collaboration diagram for ankipcpp::FieldRepository:



Additional Inherited Members

Public Member Functions inherited from `ankicpp::Repository< FieldDTO, std::int64_t >`

- `std::tuple< std::vector< FieldDTO >, bool > read ()` const
- `std::tuple< FieldDTO, bool > readById (std::int64_t key)` const
- `bool create (FieldDTO fieldDTO)` const
- `bool update (FieldDTO fieldDTO)` const
- `bool del (FieldDTO fieldDTO)` const

The documentation for this class was generated from the following file:

- `src/field/field-repository.h`

4.16 `ankicpp::GraveDTO` Class Reference

Public Attributes

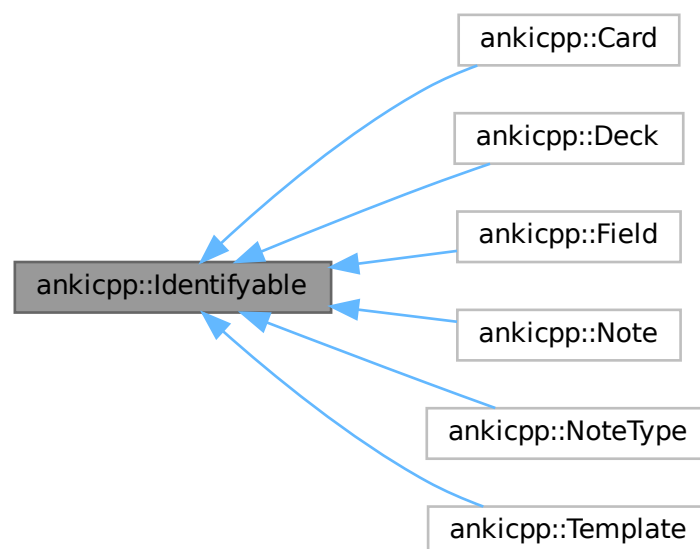
- `std::int64_t id`
- `std::int64_t type`
- `std::int64_t usn`

The documentation for this class was generated from the following file:

- `src/grave/grave-dto.h`

4.17 `ankicpp::Identifiable` Class Reference

Inheritance diagram for `ankicpp::Identifiable`:



Public Member Functions

- `std::int64_t getId ()`
- `void setId (std::int64_t id)`
- `bool operator== (Identifiable &rhs)`
- `bool operator!= (Identifiable &rhs)`
- `bool operator< (Identifiable &rhs)`

Static Public Member Functions

- `static std::int64_t generateRandomId ()`

Protected Attributes

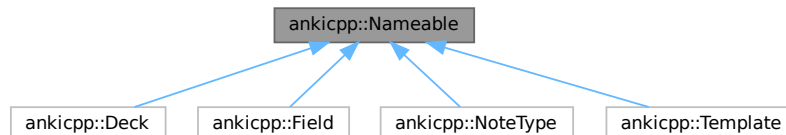
- `std::int64_t _id`

The documentation for this class was generated from the following files:

- `src/util/identifiable.h`
- `src/util/identifiable.cc`

4.18 ankicpp::Nameable Class Reference

Inheritance diagram for ankicpp::Nameable:

**Public Member Functions**

- `Nameable (std::string name)`
- `std::string getName ()`
- `void setName (std::string name)`

Protected Attributes

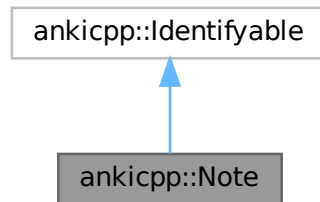
- `std::string _name`

The documentation for this class was generated from the following files:

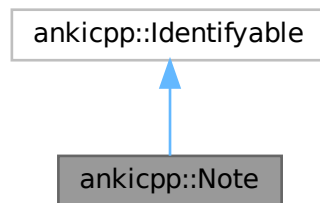
- `src/util/nameable.h`
- `src/util/nameable.cc`

4.19 ankicpp::Note Class Reference

Inheritance diagram for ankicpp::Note:



Collaboration diagram for ankicpp::Note:



Public Member Functions

- `const std::shared_ptr< NoteType > & getType () const`
- `void setType (const std::shared_ptr< NoteType > ¬eType)`
- `const std::vector< std::tuple< std::string, std::string > > & getFields () const`
- `void setFields (const std::vector< std::tuple< std::string, std::string > > &fields)`
- `std::string getField (std::string name)`
- `void addField (std::string name, std::string value)`
- `void removeField (std::string name)`
- `const std::set< std::string > & getTags () const`
- `void setTags (const std::set< std::string > &tags)`
- `void addTag (std::string name)`
- `void removeTag (std::string name)`
- `int getFlags () const`
- `void setFlags (int flags)`
- `const std::shared_ptr< Note > & getReversedParent ()`
- `void setReversedParent (const std::shared_ptr< Note > &reversedParent)`

Public Member Functions inherited from [ankipp::Identifiable](#)

- `std::int64_t getId ()`
- `void setId (std::int64_t id)`
- `bool operator== (Identifiable &rhs)`
- `bool operator!= (Identifiable &rhs)`
- `bool operator< (Identifiable &rhs)`

Additional Inherited Members

Static Public Member Functions inherited from [ankipp::Identifiable](#)

- `static std::int64_t generateRandomId ()`

Protected Attributes inherited from [ankipp::Identifiable](#)

- `std::int64_t _id`

The documentation for this class was generated from the following files:

- `src/note/note.h`
- `src/note/note.cc`

4.20 ankippp::NoteDTO Class Reference

Public Attributes

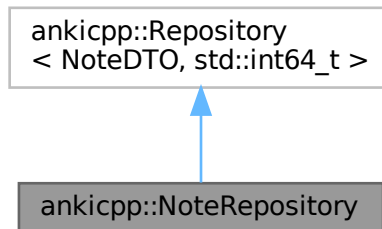
- `std::int64_t id`
- `std::string guid`
- `std::int64_t mid`
- `std::int64_t mod`
- `std::int64_t usn`
- `std::string tags`
- `std::string flds`
- `std::string sfld`
- `std::int64_t csum`
- `std::int64_t flags`
- `std::string data`

The documentation for this class was generated from the following file:

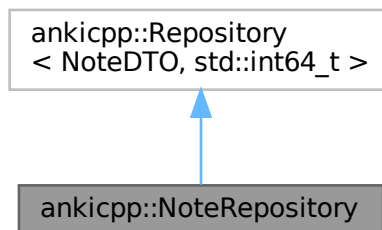
- `src/note/note-dto.h`

4.21 ankippp::NoteRepository Class Reference

Inheritance diagram for ankippp::NoteRepository:



Collaboration diagram for ankippp::NoteRepository:



Additional Inherited Members

Public Member Functions inherited from `ankipp::Repository< NoteDTO, std::int64_t >`

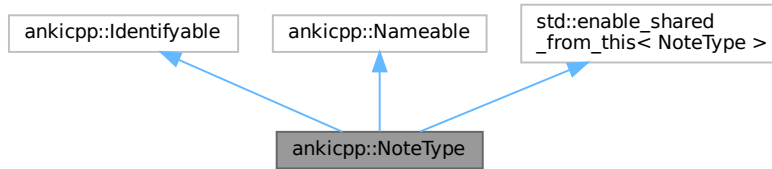
- `std::tuple< std::vector< NoteDTO >, bool > read () const`
- `std::tuple< NoteDTO, bool > readById (std::int64_t key) const`
- `bool create (NoteDTO noteDTO) const`
- `bool update (NoteDTO noteDTO) const`
- `bool del (NoteDTO noteDTO) const`

The documentation for this class was generated from the following file:

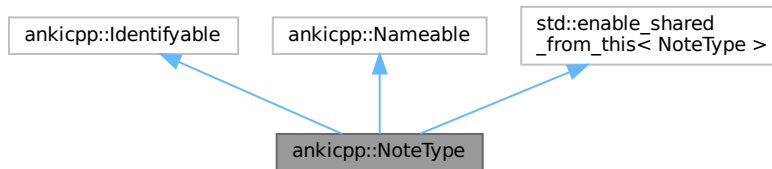
- `src/note/note-repository.h`

4.22 ankippp::NoteType Class Reference

Inheritance diagram for ankippp::NoteType:



Collaboration diagram for ankippp::NoteType:



Public Member Functions

- **NoteType** (std::string name)
- std::vector< std::shared_ptr< [Template](#) > > & **getTemplates** ()
- void **addTemplate** (std::shared_ptr< [Template](#) > templatee)
- void **removeTemplate** (std::shared_ptr< [Template](#) > templatee)
- std::size_t **getTemplateIndex** (std::shared_ptr< [Template](#) > templatee)
- std::vector< std::shared_ptr< [Field](#) > > & **getFields** ()
- void **addField** (std::shared_ptr< [Field](#) > field)
- void **removeField** (std::shared_ptr< [Field](#) > field)
- std::size_t **getFieldIndex** (std::shared_ptr< [Field](#) > field)
- std::string **getHeader** () const
- void **setHeader** (std::string header)
- std::string **getFooter** () const
- void **setFooter** (std::string footer)
- std::string **getStyle** () const
- void **setStyle** (std::string style)

Public Member Functions inherited from [ankippp::Identifiable](#)

- std::int64_t **getId** ()
- void **setId** (std::int64_t id)
- bool **operator==** ([Identifiable](#) &rhs)
- bool **operator!=** ([Identifiable](#) &rhs)
- bool **operator<** ([Identifiable](#) &rhs)

Public Member Functions inherited from [ankicpp::Nameable](#)

- **Nameable** (std::string name)
- std::string **getName** ()
- void **setName** (std::string name)

Additional Inherited Members

Static Public Member Functions inherited from [ankicpp::Identifiable](#)

- static std::int64_t **generateRandomId** ()

Protected Attributes inherited from [ankicpp::Identifiable](#)

- std::int64_t **_id**

Protected Attributes inherited from [ankicpp::Nameable](#)

- std::string **_name**

The documentation for this class was generated from the following files:

- src/note/note-type.h
- src/note/note-type.cc

4.23 [ankicpp::NoteTypeDTO](#) Class Reference

Public Attributes

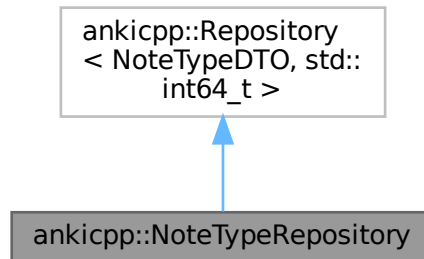
- std::int64_t **id**
- std::string **name**
- std::int64_t **mtime_secs**
- std::int64_t **usn**
- std::vector< unsigned char > **config**
- std::int64_t **cid**
- std::int64_t **ease**
- std::int64_t **ivl**
- std::int64_t **lastlvl**
- std::int64_t **factor**
- std::int64_t **time**
- std::int64_t **type**

The documentation for this class was generated from the following files:

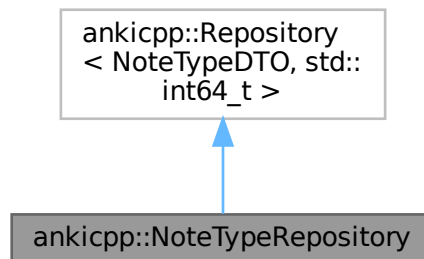
- src/note/note-type-dto.h
- src/reflog/reflog-dto.h

4.24 ankippp::NoteTypeRepository Class Reference

Inheritance diagram for ankippp::NoteTypeRepository:



Collaboration diagram for ankippp::NoteTypeRepository:



Additional Inherited Members

Public Member Functions inherited from [ankipp::Repository< NoteTypeDTO, std::int64_t >](#)

- `std::tuple< std::vector< NoteTypeDTO >, bool > read () const`
- `std::tuple< NoteTypeDTO, bool > readById (std::int64_t key) const`
- `bool create (NoteTypeDTO noteTypeDTO) const`
- `bool update (NoteTypeDTO noteTypeDTO) const`
- `bool del (NoteTypeDTO noteTypeDTO) const`

The documentation for this class was generated from the following file:

- `src/note/note-type-repository.h`

4.25 ankicpp::Repository< D, K > Class Template Reference

Public Member Functions

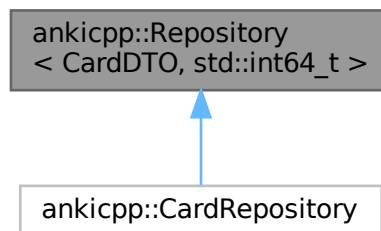
- `std::tuple< std::vector< D >, bool > read () const`
- `std::tuple< std::vector< D >, bool > readById (K key) const`
- `bool create (D dto) const`
- `bool update (D dto) const`
- `bool del (D dto) const`

The documentation for this class was generated from the following file:

- `src/database/repository.h`

4.26 ankicpp::Repository< CardDTO, std::int64_t > Class Reference

Inheritance diagram for `ankicpp::Repository< CardDTO, std::int64_t >`:



Public Member Functions

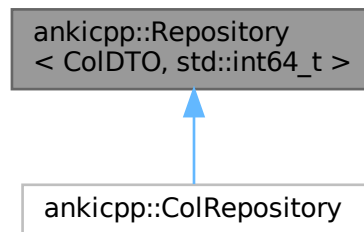
- `std::tuple< std::vector< CardDTO >, bool > read () const`
- `std::tuple< CardDTO, bool > readById (std::int64_t key) const`
- `bool create (CardDTO cardDTO) const`
- `bool update (CardDTO cardDTO) const`
- `bool del (CardDTO cardDTO) const`

The documentation for this class was generated from the following files:

- `src/card/card-repository.h`
- `src/card/card-repository.cc`

4.27 ankippp::Repository< ColDTO, std::int64_t > Class Reference

Inheritance diagram for ankippp::Repository< ColDTO, std::int64_t >:



Public Member Functions

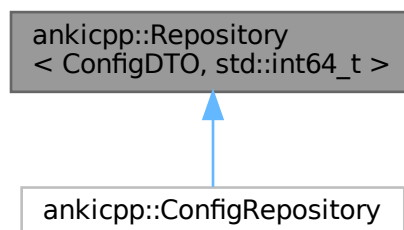
- std::tuple< std::vector< ColDTO >, bool > **read** () const
- std::tuple< ColDTO, bool > **readById** (std::int64_t key) const
- bool **create** (ColDTO colDTO) const
- bool **update** (ColDTO colDTO) const
- bool **del** (ColDTO colDTO) const

The documentation for this class was generated from the following files:

- src/col/col-repository.h
- src/col/col-repository.cc

4.28 ankippp::Repository< ConfigDTO, std::int64_t > Class Reference

Inheritance diagram for ankippp::Repository< ConfigDTO, std::int64_t >:



Public Member Functions

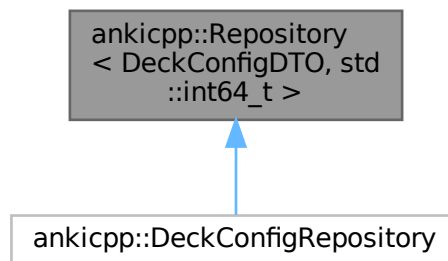
- `std::tuple< std::vector< ConfigDTO >, bool > read () const`
- `std::tuple< ConfigDTO, bool > readById (std::int64_t key) const`
- `bool create (ConfigDTO configDTO) const`
- `bool update (ConfigDTO configDTO) const`
- `bool del (ConfigDTO configDTO) const`

The documentation for this class was generated from the following files:

- `src/config/config-repository.h`
- `src/config/config-repository.cc`

4.29 `ankicpp::Repository< DeckConfigDTO, std::int64_t >` Class Reference

Inheritance diagram for `ankicpp::Repository< DeckConfigDTO, std::int64_t >`:



Public Member Functions

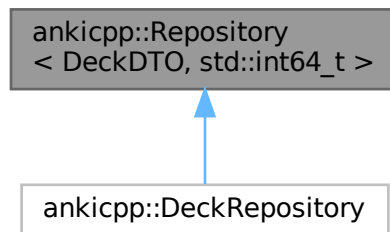
- `std::tuple< std::vector< DeckConfigDTO >, bool > read () const`
- `std::tuple< DeckConfigDTO, bool > readById (std::int64_t key) const`
- `bool create (DeckConfigDTO deckConfigDTO) const`
- `bool update (DeckConfigDTO deckConfigDTO) const`
- `bool del (DeckConfigDTO deckConfigDTO) const`

The documentation for this class was generated from the following files:

- `src/deck/deck-config-repository.h`
- `src/deck/deck-config-repository.cc`

4.30 ankippp::Repository< DeckDTO, std::int64_t > Class Reference

Inheritance diagram for ankippp::Repository< DeckDTO, std::int64_t >:



Public Member Functions

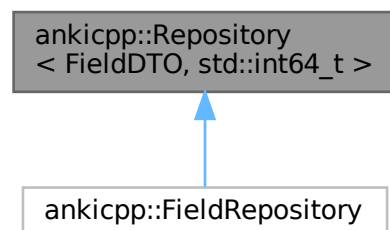
- `std::tuple< std::vector< DeckDTO >, bool > read () const`
- `std::tuple< DeckDTO, bool > readByld (std::int64_t key) const`
- `bool create (DeckDTO deckDTO) const`
- `bool update (DeckDTO deckDTO) const`
- `bool del (DeckDTO deckDTO) const`

The documentation for this class was generated from the following files:

- `src/deck/deck-repository.h`
- `src/deck/deck-repository.cc`

4.31 ankippp::Repository< FieldDTO, std::int64_t > Class Reference

Inheritance diagram for ankippp::Repository< FieldDTO, std::int64_t >:



Public Member Functions

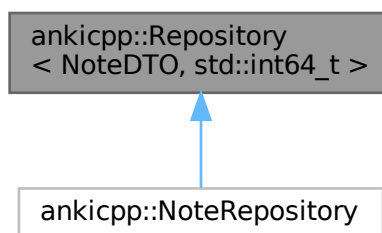
- `std::tuple< std::vector< FieldDTO >, bool > read () const`
- `std::tuple< FieldDTO, bool > readById (std::int64_t key) const`
- `bool create (FieldDTO fieldDTO) const`
- `bool update (FieldDTO fieldDTO) const`
- `bool del (FieldDTO fieldDTO) const`

The documentation for this class was generated from the following files:

- `src/field/field-repository.h`
- `src/field/field-repository.cc`

4.32 ankicpp::Repository< NoteDTO, std::int64_t > Class Reference

Inheritance diagram for `ankicpp::Repository< NoteDTO, std::int64_t >`:



Public Member Functions

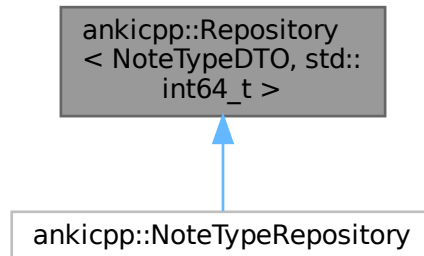
- `std::tuple< std::vector< NoteDTO >, bool > read () const`
- `std::tuple< NoteDTO, bool > readById (std::int64_t key) const`
- `bool create (NoteDTO noteDTO) const`
- `bool update (NoteDTO noteDTO) const`
- `bool del (NoteDTO noteDTO) const`

The documentation for this class was generated from the following files:

- `src/note/note-repository.h`
- `src/note/note-repository.cc`

4.33 ankippp::Repository< NoteTypeDTO, std::int64_t > Class Reference

Inheritance diagram for ankippp::Repository< NoteTypeDTO, std::int64_t >:



Public Member Functions

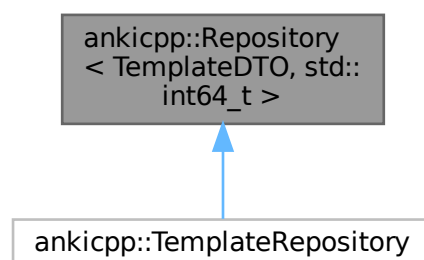
- `std::tuple< std::vector< NoteTypeDTO >, bool > read () const`
- `std::tuple< NoteTypeDTO, bool > readById (std::int64_t key) const`
- `bool create (NoteTypeDTO noteTypeDTO) const`
- `bool update (NoteTypeDTO noteTypeDTO) const`
- `bool del (NoteTypeDTO noteTypeDTO) const`

The documentation for this class was generated from the following files:

- `src/note/note-type-repository.h`
- `src/note/note-type-repository.cc`

4.34 ankippp::Repository< TemplateDTO, std::int64_t > Class Reference

Inheritance diagram for ankippp::Repository< TemplateDTO, std::int64_t >:



Public Member Functions

- `std::tuple< std::vector< TemplateDTO >, bool > read () const`
- `std::tuple< TemplateDTO, bool > readById (std::int64_t key) const`
- `bool create (TemplateDTO templateDTO) const`
- `bool update (TemplateDTO templateDTO) const`
- `bool del (TemplateDTO templateDTO) const`

The documentation for this class was generated from the following files:

- `src/template/template-repository.h`
- `src/template/template-repository.cc`

4.35 ankippp::TagDTO Class Reference

Public Attributes

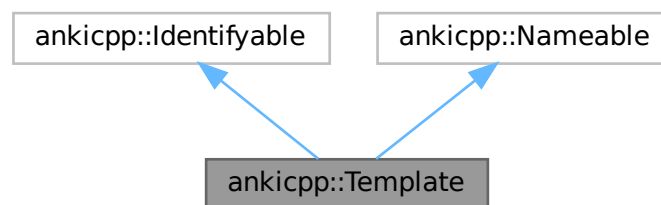
- `std::string tag`
- `std::int64_t usn`
- `bool collapsed`
- `std::vector< unsigned char > config`

The documentation for this class was generated from the following file:

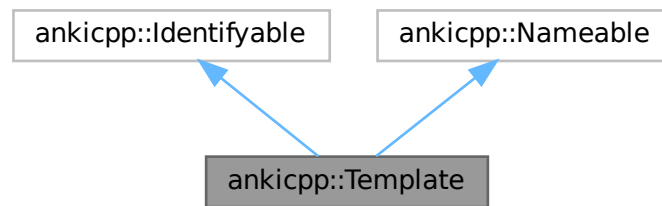
- `src/tag/tag-dto.h`

4.36 ankippp::Template Class Reference

Inheritance diagram for `ankipp::Template`:



Collaboration diagram for ankippp::Template:



Public Member Functions

- **Template** (std::string name)
- std::string **getFrontTemplate** () const
- void **setFrontTemplate** (std::string frontTemplate)
- std::string **getBackTemplate** () const
- void **setBackTemplate** (std::string backTemplate)
- std::weak_ptr< [NoteType](#) > **getNoteType** ()
- void **setNoteType** (std::weak_ptr< [NoteType](#) > noteType)

Public Member Functions inherited from [ankipp::Identifiable](#)

- std::int64_t **getId** ()
- void **setId** (std::int64_t id)
- bool **operator==** ([Identifiable](#) &rhs)
- bool **operator!=** ([Identifiable](#) &rhs)
- bool **operator<** ([Identifiable](#) &rhs)

Public Member Functions inherited from [ankipp::Nameable](#)

- **Nameable** (std::string name)
- std::string **getName** ()
- void **setName** (std::string name)

Additional Inherited Members

Static Public Member Functions inherited from [ankipp::Identifiable](#)

- static std::int64_t **generateRandomId** ()

Protected Attributes inherited from [ankipp::Identifiable](#)

- std::int64_t **_id**

Protected Attributes inherited from [ankicpp::Nameable](#)

- `std::string _name`

The documentation for this class was generated from the following files:

- `src/template/template.h`
- `src/template/template.cc`

4.37 `ankicpp::TemplateDTO` Class Reference

Public Attributes

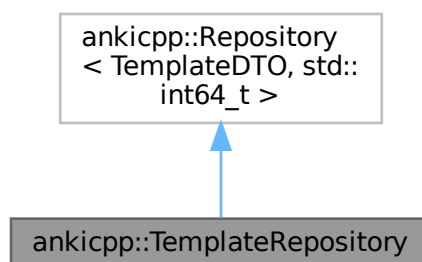
- `std::int64_t ntid`
- `std::int64_t ord`
- `std::string name`
- `std::int64_t mtime_secs`
- `std::int64_t usn`
- `std::vector< unsigned char > config`

The documentation for this class was generated from the following file:

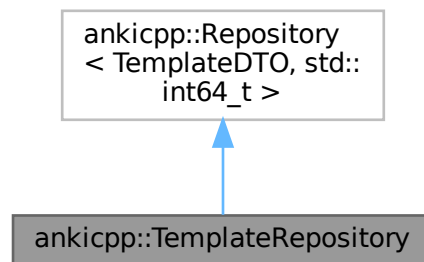
- `src/template/template-dto.h`

4.38 `ankicpp::TemplateRepository` Class Reference

Inheritance diagram for `ankicpp::TemplateRepository`:



Collaboration diagram for ankippp::TemplateRepository:



Additional Inherited Members

Public Member Functions inherited from `ankipp::Repository< TemplateDTO, std::int64_t >`

- `std::tuple< std::vector< TemplateDTO >, bool > read ()` const
- `std::tuple< TemplateDTO, bool > readById (std::int64_t key)` const
- `bool create (TemplateDTO templateDTO)` const
- `bool update (TemplateDTO templateDTO)` const
- `bool del (TemplateDTO templateDTO)` const

The documentation for this class was generated from the following file:

- `src/template/template-repository.h`

4.39 soci::type_conversion< ankippp::CardDTO > Struct Reference

Public Types

- typedef values `base_type`

Static Public Member Functions

- static void `from_base` (values const &v, indicator, [ankipp::CardDTO](#) &cardDTO)
- static void `to_base` (const [ankipp::CardDTO](#) &cardDTO, values &v, indicator &ind)

The documentation for this struct was generated from the following file:

- `src/card/card-repository.cc`

4.40 `soci::type_conversion< ankippp::ColDTO >` Struct Reference

Public Types

- typedef values `base_type`

Static Public Member Functions

- static void `from_base` (values const &v, indicator, [ankipp::ColDTO](#) &colDTO)
- static void `to_base` (const [ankipp::ColDTO](#) &colDTO, values &v, indicator &ind)

The documentation for this struct was generated from the following file:

- `src/col/col-repository.cc`

4.41 `soci::type_conversion< ankippp::ConfigDTO >` Struct Reference

Public Types

- typedef values `base_type`

Static Public Member Functions

- static void `from_base` (values const &v, indicator, [ankipp::ConfigDTO](#) &configDTO)
- static void `to_base` (const [ankipp::ConfigDTO](#) &configDTO, values &v, indicator &ind)

The documentation for this struct was generated from the following file:

- `src/config/config-repository.cc`

4.42 `soci::type_conversion< ankippp::DeckConfigDTO >` Struct Reference

Public Types

- typedef values `base_type`

Static Public Member Functions

- static void `from_base` (values const &v, indicator, [ankipp::DeckConfigDTO](#) &deckConfigDTO)
- static void `to_base` (const [ankipp::DeckConfigDTO](#) &deckConfigDTO, values &v, indicator &ind)

The documentation for this struct was generated from the following file:

- `src/deck/deck-config-repository.cc`

4.43 soci::type_conversion< ankicpp::DeckDTO > Struct Reference

Public Types

- typedef values **base_type**

Static Public Member Functions

- static void **from_base** (values const &v, indicator, [ankicpp::DeckDTO](#) &deckDTO)
- static void **to_base** (const [ankicpp::DeckDTO](#) &deckDTO, values &v, indicator &ind)

The documentation for this struct was generated from the following file:

- src/deck/deck-repository.cc

4.44 soci::type_conversion< ankicpp::FieldDTO > Struct Reference

Public Types

- typedef values **base_type**

Static Public Member Functions

- static void **from_base** (values const &v, indicator, [ankicpp::FieldDTO](#) &fieldDTO)
- static void **to_base** (const [ankicpp::FieldDTO](#) &fieldDTO, values &v, indicator &ind)

The documentation for this struct was generated from the following file:

- src/field/field-repository.cc

4.45 soci::type_conversion< ankicpp::NoteDTO > Struct Reference

Public Types

- typedef values **base_type**

Static Public Member Functions

- static void **from_base** (values const &v, indicator, [ankicpp::NoteDTO](#) ¬eDTO)
- static void **to_base** (const [ankicpp::NoteDTO](#) ¬eDTO, values &v, indicator &ind)

The documentation for this struct was generated from the following file:

- src/note/note-repository.cc

4.46 `soci::type_conversion< ankippp::NoteTypeDTO >` Struct Reference

Public Types

- typedef values **base_type**

Static Public Member Functions

- static void **from_base** (values const &v, indicator, [ankipp::NoteTypeDTO](#) ¬eTypeDTO)
- static void **to_base** (const [ankipp::NoteTypeDTO](#) ¬eTypeDTO, values &v, indicator &ind)

The documentation for this struct was generated from the following file:

- `src/note/note-type-repository.cc`

4.47 `soci::type_conversion< ankippp::TemplateDTO >` Struct Reference

Public Types

- typedef values **base_type**

Static Public Member Functions

- static void **from_base** (values const &v, indicator, [ankipp::TemplateDTO](#) &templateDTO)
- static void **to_base** (const [ankipp::TemplateDTO](#) &templateDTO, values &v, indicator &ind)

The documentation for this struct was generated from the following file:

- `src/template/template-repository.cc`

Chapter 5

File Documentation

5.1 ankicpp.h

```
00001 #pragma once
00002
00003 #include "ankicpp/config-lib.h"
00004 #include "ankicpp/export.h"
00005 #include "card/card.h"
00006 #include "compression/compression.h"
00007 #include "database/database.h"
00008 #include "deck/deck.h"
00009 #include "field/field.h"
00010 #include "note/note.h"
00011 #include "util/util.h"
00012
00013 #include <iostream>
00014
00015 namespace ankicpp {
00016 int initialized = []() {
00017     std::cout << std::format("=====\n{}@{}\n=====\n",
00018                             project_name, project_version);
00019     return 0;
00020 }();
00021 } // namespace ankicpp
```

5.2 export.h

```
00001 #pragma once
00002
00003 #include "deck/deck.h"
00004 #include <filesystem>
00005 #include <string_view>
00006 namespace ankicpp {
00007 enum class Error {
00008     EXPORT_NO_ERROR,
00009     EXPORT_UNKNOWN_ERROR,
00010     EXPORT_FILE_READING_ERROR,
00011     EXPORT_FILE_WRITING_ERROR,
00012
00013     EXPORT_COMPRESS_FILE_NOT_FOUND_ERROR,
00014     EXPORT_COMPRESS_FILE_READING_ERROR,
00015     EXPORT_COMPRESS_MINIZIP_ERROR,
00016     EXPORT_COMPRESS_ZSTD_ERROR,
00017
00018     EXPORT_DATABASE_ERROR,
00019
00020     DATA_DECK_EMPTY_ERROR,
00021     DATA_NOTE_WITHOUT_TYPE_ERROR,
00022 };
00023 extern Error error;
00024
00025 bool exportDeck(Deck deck, std::string_view filename);
00026 bool createFiles(std::filesystem::path outPath, std::filesystem::path metaPath,
00027                 std::filesystem::path mediaPath,
00028                 std::filesystem::path databasePath);
00029 bool populateDatabase(Deck deck, std::string filename);
00030 } // namespace ankicpp
```

5.3 card-dto.h

```
00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <soci/soci.h>
00005 #include <string>
00006
00007 namespace ankippp {
00008     class CardDTO {
00009     public:
00010         std::int64_t id;
00011         std::int64_t nid;
00012         std::int64_t did;
00013         std::int64_t ord;
00014         std::int64_t mod;
00015         std::int64_t usn;
00016         std::int64_t type;
00017         std::int64_t queue;
00018         std::int64_t due;
00019         std::int64_t ivl;
00020         std::int64_t factor;
00021         std::int64_t reps;
00022         std::int64_t lapses;
00023         std::int64_t left;
00024         std::int64_t odue;
00025         std::int64_t odid;
00026         std::int64_t flags;
00027         std::string data;
00028     };
00029 } // namespace ankippp
```

5.4 card-mapper.h

```
00001 #pragma once
00002
00003 #include "card/card-dto.h"
00004 #include "card/card.h"
00005 namespace ankippp {
00006     namespace cardMapper {
00007         CardDTO modelToDTO(Card &model);
00008         Card modelFromDTO(CardDTO &dto);
00009     } // namespace cardMapper
00010 } // namespace ankippp
```

5.5 card-repository.h

```
00001 #pragma once
00002
00003 #include "card/card-dto.h"
00004 #include "card/card.h"
00005 #include "database/repository.h"
00006 #include <cstdint>
00007 #include <tuple>
00008
00009 namespace ankippp {
00010     template <> class Repository<CardDTO, std::int64_t>;
00011     template <> class Repository<CardDTO, std::int64_t> {
00012     public:
00013         std::tuple<std::vector<CardDTO>, bool> read() const;
00014         std::tuple<CardDTO, bool> readById(std::int64_t key) const;
00015         bool create(CardDTO cardDTO) const;
00016         bool update(CardDTO cardDTO) const;
00017         bool del(CardDTO cardDTO) const;
00018     };
00019     class CardRepository : public Repository<CardDTO, std::int64_t> {};
00020 } // namespace ankippp
```

5.6 card.h

```
00001 #pragma once
00002
00003 #include <cstdint>
00004
00005 #include "card/card-dto.h"
```



```

00006 #include "template/template.h"
00007 #include "template/template-repository.h"
00008 #include "note/note.h"
00009 #include "util/identifiable.h"
00010
00011 namespace ankippp {
00012 class Deck;
00013 class Card: public Identifiable {
00014 public:
00015     std::shared_ptr<Deck> getDeck() const;
00016     void setDeck(std::shared_ptr<Deck> deck);
00017
00018     std::shared_ptr<Template> getTemplate() const;
00019     void setTemplate(std::shared_ptr<Template> cardType);
00020
00021     std::shared_ptr<Note> getNote() const;
00022     void setNote(std::shared_ptr<Note> note);
00023 private:
00024     std::shared_ptr<Deck> _deck;
00025     std::shared_ptr<Template> _template;
00026     std::shared_ptr<Note> _note;
00027 };
00028 } // namespace ankippp

```

5.7 col-dto.h

```

00001 #pragma once
00002
00003 #include "util/identifiable.h"
00004 #include <cstdint>
00005 #include <string>
00006 namespace ankippp {
00007 class ColDTO {
00008 public:
00009     std::int64_t id;
00010     std::int64_t crt;
00011     std::int64_t mod;
00012     std::int64_t scm;
00013     std::int64_t ver;
00014     std::int64_t dty;
00015     std::int64_t usn;
00016     std::int64_t ls;
00017     std::string conf;
00018     std::string models;
00019     std::string decks;
00020     std::string dconf;
00021     std::string tags;
00022
00023     static ColDTO createDTO() {
00024         ColDTO colDTO;
00025         colDTO.id = Identifiable::generateRandomId();
00026         colDTO.crt = 0;
00027         colDTO.mod = 0;
00028         colDTO.scm = 0;
00029         colDTO.ver = 18;
00030         colDTO.dty = 0;
00031         colDTO.usn = 0;
00032         colDTO.ls = 0;
00033         colDTO.conf = "";
00034         colDTO.models = "";
00035         colDTO.decks = "";
00036         colDTO.dconf = "";
00037         colDTO.tags = "";
00038
00039         return colDTO;
00040     }
00041 };
00042 } // namespace ankippp

```

5.8 col-repository.h

```

00001 #pragma once
00002
00003 #include "col/col-dto.h"
00004 #include "database/repository.h"
00005 #include <cstdint>
00006 #include <tuple>
00007
00008 namespace ankippp {

```

```

00009 template <> class Repository<ColDTO, std::int64_t>;
00010 template <> class Repository<ColDTO, std::int64_t> {
00011 public:
00012     std::tuple<std::vector<ColDTO>, bool> read() const;
00013     std::tuple<ColDTO, bool> readById(std::int64_t key) const;
00014     bool create(ColDTO colDTO) const;
00015     bool update(ColDTO colDTO) const;
00016     bool del(ColDTO colDTO) const;
00017 };
00018 class ColRepository : public Repository<ColDTO, std::int64_t> {};
00019 } // namespace ankipcpp

```

5.9 col.h

```

00001 #pragma once
00002
00003 #include "col/col-dto.h"

```

5.10 compression.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005
00006 namespace ankipcpp {
00007     bool compress(const std::vector<std::string> &zstdInputs,
00008                  const std::vector<std::string> &zstdOutputs,
00009                  const std::vector<std::string> &zipInputs,
00010                  const std::string &zipOutput);
00011 } // namespace ankipcpp

```

5.11 zip-utils.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005 namespace ankipcpp {
00006     namespace zip {
00007         bool compress(std::vector<std::string> inputs, std::string output);
00008     }
00009 } // namespace ankipcpp

```

5.12 zstd-utils.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005 namespace ankipcpp {
00006     namespace zstd {
00007         bool compress(std::string input, std::string output);
00008     }
00009 } // namespace ankipcpp

```

5.13 config-dto.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <string>
00005 #include <vector>
00006 namespace ankipcpp {
00007     class ConfigDTO {
00008     public:
00009         std::string KEY;

```

```
00010     std::int64_t usn;
00011     std::int64_t mtime_secs;
00012     std::vector<unsigned char> val;
00013
00014     static std::vector<ConfigDTO> createDTOs() {
00015         std::vector<ConfigDTO> configDTOs;
00016
00017         ConfigDTO configDTO;
00018
00019         configDTO.KEY = "activeDecks";
00020         configDTO.usn = 0;
00021         configDTO.mtime_secs = 0;
00022         configDTO.val.insert(configDTO.val.end(), {'l', '1', ' '});
00023         configDTOs.push_back(configDTO);
00024
00025         configDTO.KEY = "addToCur";
00026         configDTO.usn = 0;
00027         configDTO.mtime_secs = 0;
00028         configDTO.val.clear();
00029         configDTO.val.insert(configDTO.val.end(), {'t', 'r', 'u', 'e'});
00030         configDTOs.push_back(configDTO);
00031
00032         configDTO.KEY = "collapseTime";
00033         configDTO.usn = 0;
00034         configDTO.mtime_secs = 0;
00035         configDTO.val.clear();
00036         configDTO.val.insert(configDTO.val.end(), {'l', '2', '0', '0'});
00037         configDTOs.push_back(configDTO);
00038
00039         configDTO.KEY = "creationOffset";
00040         configDTO.usn = 0;
00041         configDTO.mtime_secs = 0;
00042         configDTO.val.clear();
00043         configDTO.val.insert(configDTO.val.end(), {'l', '8', '0'});
00044         configDTOs.push_back(configDTO);
00045
00046         configDTO.KEY = "curDeck";
00047         configDTO.usn = 0;
00048         configDTO.mtime_secs = 0;
00049         configDTO.val.clear();
00050         configDTO.val.insert(configDTO.val.end(), {'1'});
00051         configDTOs.push_back(configDTO);
00052
00053         configDTO.KEY = "curModel";
00054         configDTO.usn = -1;
00055         configDTO.mtime_secs = 0;
00056         configDTO.val.clear();
00057         configDTO.val.insert(configDTO.val.end(), {'0'});
00058         configDTOs.push_back(configDTO);
00059
00060         configDTO.KEY = "dayLearnFirst";
00061         configDTO.usn = 0;
00062         configDTO.mtime_secs = 0;
00063         configDTO.val.clear();
00064         configDTO.val.insert(configDTO.val.end(), {'f', 'a', 'l', 's', 'e'});
00065         configDTOs.push_back(configDTO);
00066
00067         configDTO.KEY = "dueCounts";
00068         configDTO.usn = 0;
00069         configDTO.mtime_secs = 0;
00070         configDTO.val.clear();
00071         configDTO.val.insert(configDTO.val.end(), {'t', 'r', 'u', 'e'});
00072         configDTOs.push_back(configDTO);
00073
00074         configDTO.KEY = "estTimes";
00075         configDTO.usn = 0;
00076         configDTO.mtime_secs = 0;
00077         configDTO.val.clear();
00078         configDTO.val.insert(configDTO.val.end(), {'t', 'r', 'u', 'e'});
00079         configDTOs.push_back(configDTO);
00080
00081         configDTO.KEY = "newSpread";
00082         configDTO.usn = 0;
00083         configDTO.mtime_secs = 0;
00084         configDTO.val.clear();
00085         configDTO.val.insert(configDTO.val.end(), {'0'});
00086         configDTOs.push_back(configDTO);
00087
00088         configDTO.KEY = "nextPos";
00089         configDTO.usn = 0;
00090         configDTO.mtime_secs = 0;
00091         configDTO.val.clear();
00092         configDTO.val.insert(configDTO.val.end(), {'1'});
00093         configDTOs.push_back(configDTO);
00094
00095         configDTO.KEY = "sched2021";
00096         configDTO.usn = 0;
```

```

00097     configDTO.mtime_secs = 0;
00098     configDTO.val.clear();
00099     configDTO.val.insert(configDTO.val.end(), {'t', 'r', 'u', 'e'});
00100     configDTOs.push_back(configDTO);
00101
00102     configDTO.KEY = "schedVer";
00103     configDTO.usn = 0;
00104     configDTO.mtime_secs = 0;
00105     configDTO.val.clear();
00106     configDTO.val.insert(configDTO.val.end(), {'2'});
00107     configDTOs.push_back(configDTO);
00108
00109     configDTO.KEY = "sortBackwards";
00110     configDTO.usn = 0;
00111     configDTO.mtime_secs = 0;
00112     configDTO.val.clear();
00113     configDTO.val.insert(configDTO.val.end(), {'f', 'a', 'l', 's', 'e'});
00114     configDTOs.push_back(configDTO);
00115
00116     configDTO.KEY = "sortType";
00117     configDTO.usn = 0;
00118     configDTO.mtime_secs = 0;
00119     configDTO.val.clear();
00120     configDTO.val.insert(configDTO.val.end(),
00121         {'"', 'n', 'o', 't', 'e', 'F', 'l', 'd', '"'});
00122     configDTOs.push_back(configDTO);
00123
00124     configDTO.KEY = "timeLim";
00125     configDTO.usn = 0;
00126     configDTO.mtime_secs = 0;
00127     configDTO.val.clear();
00128     configDTO.val.insert(configDTO.val.end(), {'0'});
00129     configDTOs.push_back(configDTO);
00130
00131     return configDTOs;
00132 }
00133 };
00134 } // namespace ankiapp

```

5.14 config-repository.h

```

00001 #pragma once
00002
00003 #include "config/config-dto.h"
00004 #include "database/repository.h"
00005 #include <stdint>
00006 #include <tuple>
00007
00008 namespace ankiapp {
00009 template <> class Repository<ConfigDTO, std::int64_t>;
00010 template <> class Repository<ConfigDTO, std::int64_t> {
00011 public:
00012     std::tuple<std::vector<ConfigDTO>, bool> read() const;
00013     std::tuple<ConfigDTO, bool> readById(std::int64_t key) const;
00014     bool create(ConfigDTO configDTO) const;
00015     bool update(ConfigDTO configDTO) const;
00016     bool del(ConfigDTO configDTO) const;
00017 };
00018 class ConfigRepository : public Repository<ConfigDTO, std::int64_t> {};
00019 } // namespace ankiapp

```

5.15 config.h

```

00001 #pragma once
00002
00003 #include "config/config-dto.h"

```

5.16 database.h

```

00001 #pragma once
00002
00003 #include "database/repository.h"
00004 #include "deck/deck.h"
00005 #include <functional>
00006 #include <soci/soci.h>

```

```

00007 namespace ankippp {
00008 namespace database {
00009 extern soci::session sql;
00010 bool connect(std::string filename);
00011 bool createDatabase();
00012 bool populateDatabase(Deck deck);
00013 bool disconnect();
00014 bool safeSql(std::function<void(void)> function);
00015 } // namespace database
00016 } // namespace ankippp

```

5.17 repository.h

```

00001 #pragma once
00002
00003 #include <tuple>
00004 #include <vector>
00005 namespace ankippp {
00006 template <typename D, typename K> class Repository {
00007 public:
00008     std::tuple<std::vector<D>, bool> read() const;
00009     std::tuple<std::vector<D>, bool> readById(K key) const;
00010     bool create(D dto) const;
00011     bool update(D dto) const;
00012     bool del(D dto) const;
00013 };
00014 } // namespace ankippp

```

5.18 deck-config-dto.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <string>
00005 #include <vector>
00006 namespace ankippp {
00007 class DeckConfigDTO {
00008 public:
00009     std::int64_t id;
00010     std::string name;
00011     std::int64_t mtime_secs;
00012     std::int64_t usn;
00013     std::vector<unsigned char> config;
00014
00015     static DeckConfigDTO createDefaultDTO() {
00016         DeckConfigDTO defaultConfigDTO;
00017         defaultConfigDTO.id = 1;
00018         defaultConfigDTO.name = "Default";
00019         defaultConfigDTO.mtime_secs = 0;
00020         defaultConfigDTO.usn = 0;
00021         defaultConfigDTO.config.insert(
00022             defaultConfigDTO.config.end(),
00023             {0x0a, 0x08, 0x00, 0x00, 0x80, 0x3f, 0x00, 0x00, 0x20, 0x41, 0x12,
00024              0x04, 0x00, 0x00, 0x20, 0x41, 0x22, 0x1c, 0x00, 0x00, 0x80, 0x3f,
00025              0x00, 0x00, 0x80, 0x3f, 0x00, 0x00, 0x80, 0x3f, 0x00, 0x00, 0x80,
00026              0x3f, 0x00, 0x00, 0x80, 0x3f, 0x00, 0x00, 0x80, 0x3f, 0x00, 0x00,
00027              0x80, 0x3f, 0x48, 0x14, 0x50, 0xc8, 0x01, 0x5d, 0x00, 0x00, 0x20,
00028              0x40, 0x65, 0x66, 0x66, 0xa6, 0x3f, 0x6d, 0x9a, 0x99, 0x99, 0x3f,
00029              0x7d, 0x00, 0x00, 0x80, 0x3f, 0x80, 0x01, 0x94, 0x9d, 0x02, 0x88,
00030              0x01, 0x01, 0x90, 0x01, 0x01, 0x98, 0x01, 0x04, 0xa8, 0x01, 0x01,
00031              0xb0, 0x01, 0x08, 0xc0, 0x01, 0x3c, 0xad, 0x02, 0x66, 0x66, 0x66,
00032              0x3f, 0xc5, 0x02, 0x66, 0x66, 0x66, 0x3f, 0xe0, 0x02, 0x01});
00033
00034         return defaultConfigDTO;
00035     }
00036 };
00037 } // namespace ankippp

```

5.19 deck-config-repository.h

```

00001 #pragma once
00002
00003 #include "deck/deck-config-dto.h"
00004 #include "database/repository.h"
00005 #include <cstdint>

```

```

00006 #include <tuple>
00007
00008 namespace ankippp {
00009 template <> class Repository<DeckConfigDTO, std::int64_t>;
00010 template <> class Repository<DeckConfigDTO, std::int64_t> {
00011 public:
00012     std::tuple<std::vector<DeckConfigDTO>, bool> read() const;
00013     std::tuple<DeckConfigDTO, bool> readById(std::int64_t key) const;
00014     bool create(DeckConfigDTO deckConfigDTO) const;
00015     bool update(DeckConfigDTO deckConfigDTO) const;
00016     bool del(DeckConfigDTO deckConfigDTO) const;
00017 };
00018 class DeckConfigRepository : public Repository<DeckConfigDTO, std::int64_t> {};
00019 } // namespace ankippp

```

5.20 deck-dto.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <string>
00005 #include <vector>
00006 namespace ankippp {
00007 class DeckDTO {
00008 public:
00009     std::int64_t id;
00010     std::string name;
00011     std::int64_t mtime_secs;
00012     std::int64_t usn;
00013     std::vector<unsigned char> common;
00014     std::vector<unsigned char> kind;
00015
00016     static DeckDTO createDefaultDTO() {
00017         DeckDTO deckDTO;
00018         deckDTO.id = 1;
00019         deckDTO.name = "Default";
00020         deckDTO.mtime_secs = 0;
00021         deckDTO.usn = 0;
00022
00023         deckDTO.common.insert(deckDTO.common.end(), {0x08, 0x01, 0x10, 0x01});
00024         deckDTO.kind.insert(deckDTO.kind.end(), {0x0a, 0x02, 0x08, 0x01});
00025         return deckDTO;
00026     }
00027 };
00028 } // namespace ankippp

```

5.21 deck-mapper.h

```

00001 #pragma once
00002
00003 #include "deck/deck-dto.h"
00004 #include "deck/deck.h"
00005 namespace ankippp {
00006 namespace deckMapper {
00007 DeckDTO modelToDTO(Deck &model);
00008 Deck modelFromDTO(DeckDTO &dto);
00009 } // namespace deckMapper
00010 } // namespace ankippp

```

5.22 deck-repository.h

```

00001 #pragma once
00002
00003 #include "deck/deck-dto.h"
00004 #include "deck/deck.h"
00005 #include "database/repository.h"
00006 #include <cstdint>
00007 #include <tuple>
00008
00009 namespace ankippp {
00010 template <> class Repository<DeckDTO, std::int64_t>;
00011 template <> class Repository<DeckDTO, std::int64_t> {
00012 public:
00013     std::tuple<std::vector<DeckDTO>, bool> read() const;
00014     std::tuple<DeckDTO, bool> readById(std::int64_t key) const;

```

```

00015     bool create(DeckDTO deckDTO) const;
00016     bool update(DeckDTO deckDTO) const;
00017     bool del(DeckDTO deckDTO) const;
00018 };
00019     class DeckRepository : public Repository<DeckDTO, std::int64_t> {};
00020 } // namespace ankippp

```

5.23 deck.h

```

00001 #pragma once
00002
00003 #include <memory>
00004 #include <set>
00005 #include <string>
00006
00007 #include "card/card.h"
00008 #include "deck/deck-config-dto.h"
00009 #include "deck/deck-dto.h"
00010 #include "note/note.h"
00011 #include "util/identifiable.h"
00012 #include "util/nameable.h"
00013
00014 namespace ankippp {
00015     class Deck: public Identifiable, public Nameable {
00016     public:
00017         Deck(std::string name);
00018
00019         std::set<std::shared_ptr<Note>> &getNotes();
00020         void addNote(std::shared_ptr<Note> note);
00021         void removeNote(std::shared_ptr<Note> note);
00022
00023         std::set<std::shared_ptr<Card>> &getCards();
00024         void addCard(std::shared_ptr<Card> card);
00025         void removeCard(std::shared_ptr<Card> card);
00026         void clearCards();
00027
00028         bool generateCards();
00029
00030     private:
00031         std::set<std::shared_ptr<Note>> _notes;
00032         std::set<std::shared_ptr<Card>> _cards;
00033     };
00034 } // namespace ankippp

```

5.24 field-dto.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <string>
00005 #include <vector>
00006 namespace ankippp {
00007     class FieldDTO {
00008     public:
00009         std::int64_t ntid;
00010         std::int64_t ord;
00011         std::string name;
00012         std::vector<unsigned char> config;
00013     };
00014 } // namespace ankippp

```

5.25 field-mapper.h

```

00001 #pragma once
00002
00003 #include "field/field-dto.h"
00004 #include "field/field.h"
00005 namespace ankippp {
00006     namespace fieldMapper {
00007         FieldDTO modelToDTO(Field &model);
00008         Field modelFromDTO(FieldDTO &dto);
00009     } // namespace fieldMapper
00010 } // namespace ankippp

```

5.26 field-repository.h

```

00001 #pragma once
00002
00003 #include "field/field-dto.h"
00004 #include "field/field.h"
00005 #include "database/repository.h"
00006 #include <cstdint>
00007 #include <tuple>
00008
00009 namespace ankippp {
00010 template <> class Repository<FieldDTO, std::int64_t>;
00011 template <> class Repository<FieldDTO, std::int64_t> {
00012 public:
00013     std::tuple<std::vector<FieldDTO>, bool> read() const;
00014     std::tuple<FieldDTO, bool> readById(std::int64_t key) const;
00015     bool create(FieldDTO fieldDTO) const;
00016     bool update(FieldDTO fieldDTO) const;
00017     bool del(FieldDTO fieldDTO) const;
00018 };
00019 class FieldRepository : public Repository<FieldDTO, std::int64_t> {};
00020 } // namespace ankippp

```

5.27 field.h

```

00001 #pragma once
00002
00003 #include "field/field-dto.h"
00004 #include "util/identifiable.h"
00005 #include "util/nameable.h"
00006 #include <memory>
00007 #include <string>
00008
00009 namespace ankippp {
00010 class NoteType;
00011 class Field : public Identifiable, public Nameable {
00012 public:
00013     Field(std::string name);
00014
00015     std::weak_ptr<NoteType> getNoteType() const;
00016     void setNoteType(std::weak_ptr<NoteType> noteType);
00017
00018     std::string getDescription() const;
00019     void setDescription(std::string description);
00020
00021     std::string getFont() const;
00022     void setFont(std::string font);
00023
00024     uint getFontSize() const;
00025     void setFontSize(uint fontSize);
00026
00027 private:
00028     std::weak_ptr<NoteType> _noteType;
00029     std::string _description;
00030     std::string _font;
00031     uint _fontSize;
00032 };
00033 } // namespace ankippp

```

5.28 grave-dto.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 namespace ankippp {
00005 class GraveDTO {
00006 public:
00007     std::int64_t id;
00008     std::int64_t type;
00009     std::int64_t usn;
00010 };
00011 } // namespace ankippp

```

5.29 grave.h

```

00001 #pragma once

```



```

00002
00003 #include "grave/grave-dto.h"

```

5.30 note-dto.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <string>
00005 namespace ankippp {
00006 class NoteDTO {
00007 public:
00008     std::int64_t id;
00009     std::string guid;
00010     std::int64_t mid;
00011     std::int64_t mod;
00012     std::int64_t usn;
00013     std::string tags;
00014     std::string flds;
00015     std::string sfld;
00016     std::int64_t csum;
00017     std::int64_t flags;
00018     std::string data;
00019 };
00020 } // namespace ankippp

```

5.31 note-mapper.h

```

00001 #pragma once
00002
00003 #include "note/note-dto.h"
00004 #include "note/note.h"
00005 namespace ankippp {
00006 namespace noteMapper {
00007     NoteDTO modelToDTO(Note &model);
00008     Note modelFromDTO(NoteDTO &dto);
00009 } // namespace noteMapper
00010 } // namespace ankippp

```

5.32 note-repository.h

```

00001 #pragma once
00002
00003 #include "note/note-dto.h"
00004 #include "note/note.h"
00005 #include "database/repository.h"
00006 #include <cstdint>
00007 #include <tuple>
00008
00009 namespace ankippp {
00010 template <> class Repository<NoteDTO, std::int64_t>;
00011 template <> class Repository<NoteDTO, std::int64_t> {
00012 public:
00013     std::tuple<std::vector<NoteDTO>, bool> read() const;
00014     std::tuple<NoteDTO, bool> readById(std::int64_t key) const;
00015     bool create(NoteDTO noteDTO) const;
00016     bool update(NoteDTO noteDTO) const;
00017     bool del(NoteDTO noteDTO) const;
00018 };
00019 class NoteRepository : public Repository<NoteDTO, std::int64_t> {};
00020 } // namespace ankippp

```

5.33 note-type-dto.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <string>
00005 #include <vector>
00006 namespace ankippp {
00007 class NoteTypeDTO {

```

```

00008 public:
00009     std::int64_t id;
00010     std::string name;
00011     std::int64_t mtime_secs;
00012     std::int64_t usn;
00013     std::vector<unsigned char> config;
00014 };
00015 } // namespace ankippp

```

5.34 note-type-mapper.h

```

00001 #pragma once
00002
00003 #include "note/note-type-dto.h"
00004 #include "note/note-type.h"
00005 namespace ankippp {
00006     namespace noteTypeMapper {
00007         NoteTypeDTO modelToDTO(NoteType &model);
00008         NoteType modelFromDTO(NoteTypeDTO &dto);
00009     } // namespace noteTypeMapper
00010 } // namespace ankippp

```

5.35 note-type-repository.h

```

00001 #pragma once
00002
00003 #include "note/note-type.h"
00004 #include "note/note-type-dto.h"
00005 #include "database/repository.h"
00006 #include <cstdint>
00007 #include <tuple>
00008
00009 namespace ankippp {
00010     template< > class Repository<NoteTypeDTO, std::int64_t>;
00011     template< > class Repository<NoteTypeDTO, std::int64_t> {
00012     public:
00013         std::tuple<std::vector<NoteTypeDTO>, bool> read() const;
00014         std::tuple<NoteTypeDTO, bool> readById(std::int64_t key) const;
00015         bool create(NoteTypeDTO noteTypeDTO) const;
00016         bool update(NoteTypeDTO noteTypeDTO) const;
00017         bool del(NoteTypeDTO noteTypeDTO) const;
00018     };
00019     class NoteTypeRepository : public Repository<NoteTypeDTO, std::int64_t> {};
00020 } // namespace ankippp

```

5.36 note-type.h

```

00001 #pragma once
00002
00003 #include <memory>
00004 #include <string>
00005
00006 #include "field/field.h"
00007 #include "template/template.h"
00008 #include "util/identifiable.h"
00009 #include "util/nameable.h"
00010
00011 namespace ankippp {
00012     class NoteType : public Identifiable,
00013                     public Nameable,
00014                     public std::enable_shared_from_this<NoteType> {
00015     public:
00016         NoteType(std::string name);
00017
00018         std::vector<std::shared_ptr<Template>> &getTemplates();
00019         void addTemplate(std::shared_ptr<Template> templatee);
00020         void removeTemplate(std::shared_ptr<Template> templatee);
00021         std::size_t getTemplateIndex(std::shared_ptr<Template> templatee);
00022
00023         std::vector<std::shared_ptr<Field>> &getFields();
00024         void addField(std::shared_ptr<Field> field);
00025         void removeField(std::shared_ptr<Field> field);
00026         std::size_t getFieldIndex(std::shared_ptr<Field> field);
00027
00028         std::string getHeader() const;

```

```

00029 void setHeader(std::string header);
00030
00031 std::string getFooter() const;
00032 void setFooter(std::string footer);
00033
00034 std::string getStyle() const;
00035 void setStyle(std::string style);
00036
00037 private:
00038     std::vector<std::shared_ptr<Template>> _templates;
00039     std::vector<std::shared_ptr<Field>> _fields;
00040     std::string _header;
00041     std::string _footer;
00042     std::string _style;
00043 };
00044
00045 extern const std::shared_ptr<Template> basicTemplate;
00046 extern const std::shared_ptr<Template> reversedTemplate;
00047 extern const std::shared_ptr<NoteType> basicNoteType;
00048 extern const std::shared_ptr<NoteType> basicAndReversedNoteType;
00049
00050 extern const std::shared_ptr<Template> clozeTemplate;
00051 extern const std::shared_ptr<NoteType> clozeNoteType;
00052 } // namespace ankippp

```

5.37 note.h

```

00001 #pragma once
00002
00003 #include <memory>
00004 #include <set>
00005 #include <string>
00006
00007 #include "note/note-type.h"
00008 #include "util/identifyable.h"
00009
00010 namespace ankippp {
00011     class Note : public Identifyable {
00012     public:
00013         const std::shared_ptr<NoteType> &getType() const;
00014         void setType(const std::shared_ptr<NoteType> &noteType);
00015
00016         const std::vector<std::tuple<std::string, std::string>> &getFields() const;
00017         void setFields(const std::vector<std::tuple<std::string, std::string>> &fields);
00018         std::string getField(std::string name);
00019         void addField(std::string name, std::string value);
00020         void removeField(std::string name);
00021
00022         const std::set<std::string> &getTags() const;
00023         void setTags(const std::set<std::string> &tags);
00024         void addTag(std::string name);
00025         void removeTag(std::string name);
00026
00027         int getFlags() const;
00028         void setFlags(int flags);
00029
00030         const std::shared_ptr<Note> &getReversedParent();
00031         void setReversedParent(const std::shared_ptr<Note> &reversedParent);
00032     private:
00033         std::shared_ptr<NoteType> _noteType;
00034         std::vector<std::tuple<std::string, std::string>> _fields;
00035         std::set<std::string> _tags;
00036         int _flags = 0;
00037         std::shared_ptr<Note> _reversedParent;
00038     };
00039 } // namespace ankippp

```

5.38 relog-dto.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 namespace ankippp {
00005     class NoteTypeDTO {
00006     public:
00007         std::int64_t id;
00008         std::int64_t cid;
00009         std::int64_t usn;
00010         std::int64_t ease;

```

```
00011     std::int64_t ivl;
00012     std::int64_t lastIvl;
00013     std::int64_t factor;
00014     std::int64_t time;
00015     std::int64_t type;
00016 };
00017 } // namespace ankippp
```

5.39 reflog.h

```
00001 #pragma once
00002
00003 #include "reflog/reflog-dto.h"
```

5.40 tag-dto.h

```
00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <string>
00005 #include <vector>
00006 namespace ankippp {
00007     class TagDTO {
00008     public:
00009         std::string tag;
00010         std::int64_t usn;
00011         bool collapsed;
00012         std::vector<unsigned char> config;
00013     };
00014 } // namespace ankippp
```

5.41 tag.h

```
00001 #pragma once
00002
00003 #include "tag/tag-dto.h"
```

5.42 template-dto.h

```
00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <soci/soci.h>
00005 #include <string>
00006 #include <vector>
00007 namespace ankippp {
00008     class TemplateDTO {
00009     public:
00010         std::int64_t ntid;
00011         std::int64_t ord;
00012         std::string name;
00013         std::int64_t mtime_secs;
00014         std::int64_t usn;
00015         std::vector<unsigned char> config;
00016     };
00017 } // namespace ankippp
```

5.43 template-mapper.h

```
00001 #pragma once
00002
00003 #include "template/template.h"
00004 namespace ankippp {
00005     namespace templateMapper {
00006         TemplateDTO modelToDTO(Template &model);
00007         Template modelFromDTO(TemplateDTO &dto);
00008     } // namespace templateMapper
00009 } // namespace ankippp
```

5.44 template-repository.h

```

00001 #pragma once
00002
00003 #include "database/repository.h"
00004 #include "template/template-dto.h"
00005 #include <tuple>
00006
00007 namespace ankippp {
00008
00009 template <> class Repository<TemplateDTO, std::int64_t>;
00010
00011 template <> class Repository<TemplateDTO, std::int64_t> {
00012 public:
00013     std::tuple<std::vector<TemplateDTO>, bool> read() const;
00014     std::tuple<TemplateDTO, bool> readById(std::int64_t key) const;
00015     bool create(TemplateDTO templatedDTO) const;
00016     bool update(TemplateDTO templatedDTO) const;
00017     bool del(TemplateDTO templatedDTO) const;
00018 };
00019
00020 class TemplateRepository : public Repository<TemplateDTO, std::int64_t> {};
00021
00022 } // namespace ankippp

```

5.45 template.h

```

00001 #pragma once
00002
00003 #include "template/template-dto.h"
00004 #include "template/template-repository.h"
00005 #include "util/identifiable.h"
00006 #include "util/nameable.h"
00007 #include <cstdint>
00008 #include <memory>
00009 #include <string>
00010
00011 namespace ankippp {
00012 class NoteType;
00013 class Template : public Identifiable, public Nameable {
00014 public:
00015     Template(std::string name);
00016
00017     std::string getFrontTemplate() const;
00018     void setFrontTemplate(std::string frontTemplate);
00019
00020     std::string getBackTemplate() const;
00021     void setBackTemplate(std::string backTemplate);
00022
00023     std::weak_ptr<NoteType> getNoteType();
00024     void setNoteType(std::weak_ptr<NoteType> noteType);
00025
00026 private:
00027     std::string _frontTemplate;
00028     std::string _backTemplate;
00029     std::weak_ptr<NoteType> _noteType;
00030 };
00031 } // namespace ankippp

```

5.46 identifiable.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 namespace ankippp {
00005 class Identifiable {
00006 public:
00007     Identifiable();
00008
00009     std::int64_t getId();
00010     void setId(std::int64_t id);
00011
00012     static std::int64_t generateRandomId();
00013
00014     bool operator==(Identifiable &rhs);
00015     bool operator!=(Identifiable &rhs);
00016     bool operator<(Identifiable &rhs);
00017
00018 protected:

```

```
00019     std::int64_t _id;
00020 };
00021 } // namespace ankicpp
```

5.47 nameable.h

```
00001 #pragma once
00002
00003 #include <string>
00004 namespace ankicpp {
00005     class Nameable {
00006     public:
00007         Nameable();
00008         Nameable(std::string name);
00009
00010         std::string getName();
00011         void setName(std::string name);
00012     protected:
00013         std::string _name;
00014     };
00015 } // namespace ankicpp
```

5.48 time.h

```
00001 #pragma once
00002
00003 #include <chrono>
00004 #include <cstdint>
00005
00006 namespace ankicpp {
00007     int64_t getNow();
00008 }
```

5.49 util.h

```
00001 #pragma once
00002
00003 #include "util/time.h"
00004 #include "util/nameable.h"
00005 #include "util/identifiable.h"
```

Index

ankicpp::Card, [7](#)
ankicpp::CardDTO, [8](#)
ankicpp::CardRepository, [9](#)
ankicpp::ColDTO, [10](#)
ankicpp::ColRepository, [10](#)
ankicpp::ConfigDTO, [11](#)
ankicpp::ConfigRepository, [12](#)
ankicpp::Deck, [13](#)
ankicpp::DeckConfigDTO, [14](#)
ankicpp::DeckConfigRepository, [15](#)
ankicpp::DeckDTO, [16](#)
ankicpp::DeckRepository, [16](#)
ankicpp::Field, [17](#)
ankicpp::FieldDTO, [19](#)
ankicpp::FieldRepository, [19](#)
ankicpp::GraveDTO, [20](#)
ankicpp::Identifiable, [20](#)
ankicpp::Nameable, [21](#)
ankicpp::Note, [22](#)
ankicpp::NoteDTO, [23](#)
ankicpp::NoteRepository, [24](#)
ankicpp::NoteType, [25](#)
ankicpp::NoteTypeDTO, [26](#)
ankicpp::NoteTypeRepository, [27](#)
ankicpp::Repository< CardDTO, std::int64_t >, [28](#)
ankicpp::Repository< ColDTO, std::int64_t >, [29](#)
ankicpp::Repository< ConfigDTO, std::int64_t >, [29](#)
ankicpp::Repository< D, K >, [28](#)
ankicpp::Repository< DeckConfigDTO, std::int64_t >, [30](#)
ankicpp::Repository< DeckDTO, std::int64_t >, [31](#)
ankicpp::Repository< FieldDTO, std::int64_t >, [31](#)
ankicpp::Repository< NoteDTO, std::int64_t >, [32](#)
ankicpp::Repository< NoteTypeDTO, std::int64_t >, [33](#)
ankicpp::Repository< TemplateDTO, std::int64_t >, [33](#)
ankicpp::TagDTO, [34](#)
ankicpp::Template, [34](#)
ankicpp::TemplateDTO, [36](#)
ankicpp::TemplateRepository, [36](#)

soci::type_conversion< ankicpp::CardDTO >, [37](#)
soci::type_conversion< ankicpp::ColDTO >, [38](#)
soci::type_conversion< ankicpp::ConfigDTO >, [38](#)
soci::type_conversion< ankicpp::DeckConfigDTO >, [38](#)
soci::type_conversion< ankicpp::DeckDTO >, [39](#)
soci::type_conversion< ankicpp::FieldDTO >, [39](#)
soci::type_conversion< ankicpp::NoteDTO >, [39](#)
soci::type_conversion< ankicpp::NoteTypeDTO >, [40](#)
soci::type_conversion< ankicpp::TemplateDTO >, [40](#)
src/ankicpp/ankicpp.h, [41](#)
src/ankicpp/export.h, [41](#)
src/card/card-dto.h, [42](#)
src/card/card-mapper.h, [42](#)
src/card/card-repository.h, [42](#)
src/card/card.h, [42](#)
src/col/col-dto.h, [43](#)
src/col/col-repository.h, [43](#)
src/col/col.h, [44](#)
src/compression/compression.h, [44](#)
src/compression/zip-utils.h, [44](#)
src/compression/zstd-utils.h, [44](#)
src/config/config-dto.h, [44](#)
src/config/config-repository.h, [46](#)
src/config/config.h, [46](#)
src/database/database.h, [46](#)
src/database/repository.h, [47](#)
src/deck/deck-config-dto.h, [47](#)
src/deck/deck-config-repository.h, [47](#)
src/deck/deck-dto.h, [48](#)
src/deck/deck-mapper.h, [48](#)
src/deck/deck-repository.h, [48](#)
src/deck/deck.h, [49](#)
src/field/field-dto.h, [49](#)
src/field/field-mapper.h, [49](#)
src/field/field-repository.h, [50](#)
src/field/field.h, [50](#)
src/grave/grave-dto.h, [50](#)
src/grave/grave.h, [50](#)
src/note/note-dto.h, [51](#)
src/note/note-mapper.h, [51](#)
src/note/note-repository.h, [51](#)
src/note/note-type-dto.h, [51](#)
src/note/note-type-mapper.h, [52](#)
src/note/note-type-repository.h, [52](#)
src/note/note-type.h, [52](#)
src/note/note.h, [53](#)
src/reflog/reflog-dto.h, [53](#)
src/reflog/reflog.h, [54](#)
src/tag/tag-dto.h, [54](#)
src/tag/tag.h, [54](#)
src/template/template-dto.h, [54](#)
src/template/template-mapper.h, [54](#)
src/template/template-repository.h, [55](#)
src/template/template.h, [55](#)
src/util/identifiable.h, [55](#)
src/util/nameable.h, [56](#)
src/util/time.h, [56](#)
src/util/util.h, [56](#)